Structural Features Dominate Current Approaches to Progress Stagnation Detection in LLM Agents: A Systematic Empirical Study with Human Validation

Anonymous

Abstract

LLM-based agents frequently enter *progress stagnation*—sustained periods of goal-irrelevant activity that waste compute and degrade task outcomes. We present a systematic empirical study of stagnation detection through weak supervision on 54,809 trajectory windows from two coding agents (SWE-Agent and OpenHands). Our central finding, validated on a 650-window human-annotated gold set ($\kappa = 0.67$), is that **structural features dominate all current automated approaches**: a Gradient Boosted classifier over 30 lightweight features (command repetition ratio, file diff frequency, unique command fraction) achieves macro-F1 = 0.76 on gold labels, outperforming neural embedding probes (0.69–0.72) and an LLM-as-judge baseline (0.60). However, gold evaluation reveals that labeling-function-derived labels inflate performance by $\sim$22 percentage points through circular evaluation, and that 28.9% of human-recognized stagnation constitutes a *semantic residual*—strategic dead-ends, implicit repetition, and unfocused exploration—that no current method captures effectively (F1 $\leq$ 0.38). We further show that labeling functions exhibit systematic but agent-specific temporal dynamics, with onset alignment to human annotations at 52% within $\pm$2 steps—a statistical trend, not a precise predictor. Finally, three temporal label model implementations (TWV, HMM, lag-aware HMM) all fail to improve over majority vote, with TWV degrading downstream classifiers by up to 21.7pp. Our results redirect agent monitoring research from expensive representation learning toward lightweight structural approaches, while identifying semantic stagnation as a concrete open challenge.

1 Introduction

LLM-based agents deployed on complex software engineering tasks—such as those benchmarked by SWE-bench—routinely enter prolonged episodes of unproductive behavior. An agent may issue diverse, syntactically varied commands that individually appear reasonable yet collectively make no progress toward the goal. This *progress stagnation* phenomenon wastes compute, delays task resolution, and represents a key bottleneck for autonomous agent deployments.

Existing detection approaches fall into three broad categories, each with significant limitations. *Cycle detectors* identify literal command or state repetition but miss semantically diverse stagnation. *Process reward models* (PRMs) provide step-level value estimates but require outcome labels and RL training pipelines, positioning them as training signals rather than monitoring tools. *Self-reflection methods* (Reflexion, LATS) rely on the stagnating agent's own judgment—precisely the capability that has failed.

We present a systematic empirical study of stagnation detection through weak supervision. Using three labeling functions (LFs)—command repetition, file diff inactivity, and outcome backpropagation—we label 54,809 trajectory windows from SWE-Agent and OpenHands, then train and compare struc-

tural classifiers, neural embedding probes, and an LLM-as-judge baseline. Crucially, we validate all findings against a 650-window human-annotated gold set.

Our study yields three contributions:

1. **Structural features dominate current approaches** (§5). On gold labels, a Gradient Boosted structural classifier (macro-F1 = 0.76) outperforms the best neural probe (0.72, BGE-m3 encoder) by 4pp and an LLM-as-judge (0.60) by 16pp. LF-approximated stagnation is structural in nature; however, human-recognizable stagnation has additional semantic dimensions that no current automated method captures. Gold evaluation further reveals that LF-derived evaluation inflates reported performance by ∼22pp through circular evaluation—a methodological finding with implications for any weak supervision study on sequential data.
2. **LF temporal dynamics are systematic but noisy** (§6). LFs exhibit agent-specific temporal roles: on SWE-Agent, `cmd_rep` leads stagnation onset by 1.2 steps (gold-validated) while `outcome_bp` lags by 5.3 steps. Onset alignment with human annotations reaches 52% within ±2 steps—a useful statistical trend (±2–3 step noise) but not a precise intervention trigger.
3. **Temporal label models fail to improve over majority vote** (§9). Three implementations (TWV, HMM, lag-aware HMM) all fail to surpass simple majority vote, with TWV labels degrading downstream structural classifiers by up to 21.7pp in cross-agent transfer. This well-motivated negative finding constrains future temporal label model research by identifying the low-conflict regime (≥95% agreement at conf≥2) as a key limiting factor.

2 Related Work

Agent Failure Detection. Cycle detection methods (CDCS+CDSA, SpecRA, TrajAD) identify repetitive or anomalous agent states but miss semantically diverse stagnation where commands vary in surface form. SlopCodeBench quantifies coding agent degradation over long trajectories, reporting verbosity increases in 90% of trajectories and quality erosion in 80%, motivating the need for real-time monitoring.

Process Reward Models. AgentPRM trains step-level PRMs using TD/GAE to evaluate "promise and progress," but requires outcome labels and RL infrastructure. Our approach differs fundamentally: we use weak supervision (no outcome labels), operate at the window level (capturing temporal dynamics), and target monitoring rather than training.

Weak Supervision on Sequential Data. The data programming paradigm, originating from Snorkel, assumes conditionally independent labeling functions. Agent trajectories violate this assumption: LFs fire in systematic temporal patterns along trajectories. Our study empirically characterizes this violation and tests whether temporal label models can exploit it.

Self-Reflection and LLM-as-Judge. Reflexion, LATS, and Tree of Thoughts use LLM self-evaluation for course correction. These are intervention mechanisms, not monitoring tools, and rely on the stagnating agent's own (degraded) judgment. LLM-as-judge approaches offer external evaluation but at high computational cost.

Table 1: Gold set annotation summary.

Metric	Value
Total windows	650
Inter-annotator κ	0.67
Inter-annotator % agreement	81%
Gold stagnation ratio	41.5%
LF stagnation ratio (same sample)	33.5%
Adjudicated windows	124 (19.1%)

Table 2: LF label accuracy vs. gold annotations by confidence tier.

Confidence	N	Accuracy	κ	$\text{Prec}_{\text{stag}}$	Rec_{stag}
conf=3	60	0.92	0.83	0.94	0.88
conf=2	340	0.82	0.62	0.84	0.73
conf=1	250	0.51	0.08	0.55	0.41
Overall	650	0.72	0.61	0.78	0.63

3 Problem Setting and Data

Task and Agents. We study SWE-bench Verified trajectories from two architecturally distinct coding agents: SWE-Agent (command-line interaction, structured action space) and OpenHands (IDE-based, broader action repertoire). Trajectories are segmented into overlapping windows of $k=5$ consecutive steps.

Labeling Functions. Three LFs provide complementary stagnation signals:

- `cmd_rep`: Command repetition ratio within the window, thresholded.
- `file_diff`: File modification inactivity—no meaningful file changes across the window.
- `outcome_bp`: Outcome backpropagation—stagnation label propagated backward from unsuccessful task completion.

Majority vote (MV) over these three LFs produces the training label. Each window receives a confidence score (1–3) equal to the number of agreeing LFs. The full dataset comprises 54,809 labeled windows. At conf$\geq$2, the LF agreement rate is 95.1%.

Gold Annotation. We construct a 650-window gold set via stratified sampling (Table 1). Two annotators independently label each window; a third adjudicates disagreements ($n=124$, 19.1%). Inter-annotator agreement reaches $\kappa=0.67$ (substantial), with disagreements concentrated on edge cases: ineffective edits and late-trajectory exploratory attempts.

The gold stagnation ratio (41.5%) significantly exceeds the LF ratio (33.5%), indicating that LFs systematically under-report stagnation—particularly cases where agents issue diverse commands but make no semantic progress.

4 LF Label Validation Against Gold

Table 2 reports LF label accuracy against gold annotations, stratified by confidence.

Table 3: LF vs. gold confusion matrix (overall, $N=650$).

	Gold=Stag	Gold=Prog
LF=Stag	170 (TP)	48 (FP)
LF=Prog	100 (FN)	332 (TN)

At conf$\geq$2, LF labels achieve 82% accuracy ($\kappa=0.62$), validating their use as training signal for this tier. However, conf=1 windows (72% of the full dataset: 39,408 of 54,809) have near-random accuracy (0.51, $\kappa=0.08$), explaining why models trained on all data underperform those trained on conf$\geq$2 subsets.

Table 3 shows the overall confusion matrix. LF precision for stagnation is reasonably high (0.78)—when LFs flag stagnation, they are usually correct—but recall is low (0.63), systematically missing 37% of human-annotated stagnation. These missed cases constitute the *semantic residual* analyzed in §7.

The dominant false-positive pattern is `cmd_rep` firing on productive-but-repetitive debugging sequences. The dominant false-negative pattern is semantic stagnation with diverse commands—LFs see activity, humans see no progress.

5 Structural Features Dominate Current Approaches

5.1 Models

We compare three model families on the gold set:

- **Structural classifiers**: Gradient Boosted (GB), Random Forest (RF), and Logistic Regression (LR) over 30 lightweight features (command repetition ratio, unique command fraction, file diff counts, edit sizes, error frequencies, etc.). Inference time: <5ms per window.
- **Neural probes**: GRU-based sequence classifiers operating on frozen embeddings from three encoders (Qwen3-0.6B, E5-large-v2, BGE-m3).
- **LLM-as-judge**: Claude Sonnet, zero-shot prompting on window summaries.

All models are trained on LF-derived labels (conf$\geq$2 subset) and evaluated on the 650-window gold set.

5.2 Gold Evaluation Results

Table 4 reports gold-set performance alongside the original LF-evaluated scores, revealing the magnitude of circular evaluation inflation.

Circular evaluation inflation. Structural classifiers drop $\sim$22–26pp from LF-eval to gold-eval, while the LLM-as-judge drops only 6.3pp. This asymmetry is expected: structural features are direct inputs to the LFs that generated the training labels, so near-perfect LF-eval scores reflect reverse-engineering of the labeling rules rather than genuine detection capability. The LLM-as-judge, never fitted to LF labels, is least affected.

Table 4: Model performance on gold labels vs. LF-derived labels, showing circular evaluation inflation (Δ).

Model	LF-eval F1	Gold-eval F1	Δ (pp)
Structural GB	0.995	0.76	−23.5
Structural RF	0.995	0.74	−25.5
Structural LR	0.987	0.73	−25.7
Probe (GRU, Qwen3)	0.906	0.69	−21.6
LLM-as-judge	0.663	0.60	−6.3

Table 5: Probe variants on gold labels by encoder.

Encoder	LF-eval F1	Gold-eval F1	Δ vs. Struct. GB
Qwen3-0.6B	0.906	0.69	−7pp
E5-large-v2	0.936	0.71	−5pp
BGE-m3	0.941	0.72	−4pp

Structural features still dominate. Despite the deflation, structural GB (0.76) remains the best performer on gold labels, outperforming the best neural probe (BGE-m3, 0.72) by 4pp and the LLM-as-judge (0.60) by 16pp. On the conf$\geq$2 gold subset ($n = 400$), structural GB reaches 0.82, demonstrating strong performance where LF labels are reliable. Even on conf=1 windows (n=250), structural GB (0.67) exceeds the LF accuracy (0.51) by 16pp, indicating that the model learns smoother decision boundaries beyond individual LF rules.

5.3 Encoder Comparison for Neural Probes

Table 5 compares probe variants by encoder.

BGE-m3 emerges as the strongest probe encoder on gold labels (0.72), despite smaller margins on LF-eval. This ranking reversal suggests that encoder generalization differences are compressed by circular evaluation and only surface on independent gold labels. The structural–probe gap narrows from 8.96pp (LF-eval) to 7pp (gold), indicating that embeddings capture marginal semantic signal invisible to structural features—but not enough to close the gap.

5.4 LLM-as-Judge Error Decomposition

Table 6 decomposes LLM-judge errors relative to LF labels on the gold set.

The 8.2% of windows where the LLM is correct but LFs are wrong represent cases where semantic understanding adds genuine value—typically "active but ineffective" patterns where the agent edits files but the edits are logically flawed. However, the LLM errs more often than it helps (14.5% wrong vs. 8.2% uniquely correct), often misclassifying verbose error output as stagnation. The net effect: the LLM-as-judge's independent semantic judgment is insufficiently reliable for monitoring, though it identifies real failure modes that structural approaches miss.

Table 6: LLM-judge vs. LF error decomposition on gold ($N = 650$).

Category	Count	%
Both correct	398	61.2%
LLM correct, LF wrong	53	8.2%
LLM wrong, LF correct	94	14.5%
Both wrong	40	6.1%
Ambiguous (adjudicated)	65	10.0%

Table 7: LF onset alignment with gold annotations ($N = 270$ stagnation windows).

LF	Mean Offset (steps)	Within ±2	Within ±3
`cmd_rep`	−1.2	48%	61%
`file_diff`	−0.6	56%	68%
`outcome_bp`	+5.3	22%	34%
Any LF (first fire)	−0.8	52%	64%

6 LF Temporal Dynamics

6.1 Onset Alignment with Gold Annotations

Table 7 reports alignment between LF first-fire and human-annotated stagnation onset across the 270 gold stagnation windows.

`file_diff` shows the best alignment with human-perceived onset (56% within ±2 steps, mean offset −0.6), consistent with "no file changes" being an intuitive stagnation signal shared by humans and LFs. `cmd_rep` fires slightly early (−1.2 steps), detecting structural repetition before humans perceive stagnation. `outcome_bp` lags significantly (+5.3 steps) by design, as it propagates outcome information backward.

Overall onset alignment of 52% within ±2 steps confirms that LF temporal dynamics capture a real statistical trend, but with substantial noise (±2–3 steps, $\sigma = 2.1$). These findings support Claim 2 with the caveat that onset timing is a coarse indicator rather than a precise intervention trigger.

6.2 Agent-Specific Temporal Roles

On SWE-Agent trajectories (from LF-eval data), `cmd_rep` leads stagnation onset by −1.87 steps while on OpenHands it reverses to a lagging role (+3.32 steps). This role reversal reflects architectural differences: OpenHands' broader action repertoire reduces literal command repetition, making `cmd_rep` fire late (92.3% firing rate, 71% false positive rate on OH). Ablation confirms that removing `cmd_rep` from the OH labeling pipeline improves F1 by +21.6pp, demonstrating that LF behavior is not transferable across agent architectures without recalibration.

7 Stagnation Type Taxonomy

Error analysis on the gold set reveals a taxonomy of stagnation types (Table 8).

Structural stagnation (71.1%). The majority of gold stagnation is *structural*: repetitive command patterns (54.1%) and low file activity (17.0%). All methods detect these effectively, with

Table 8: Stagnation type taxonomy on gold set ($N = 270$ stagnation windows).

Type	N (%)	Struct. F1	Probe F1	LLM F1
Repetitive-structural	146 (54.1%)	0.88	0.79	0.58
Low-activity stagnation	46 (17.0%)	0.83	0.72	0.55
Strategic dead-end	31 (11.5%)	0.22	0.28	0.38
Implicit repetition	24 (8.9%)	0.31	0.35	0.33
Unfocused exploration	23 (8.5%)	0.18	0.24	0.30

Table 9: $F1_{stag}$ breakdown by dimension (LF test set, conf$\geq$2).

Dimension	Subcategory	N_{stag}	$F1_{stag}$	Δ vs. overall
Position	Early (0–33%)	98	0.88	+2.1pp
	Middle (33–66%)	132	0.79	−6.9pp
	Late (66–100%)	255	0.87	+1.1pp
Dom. LF	`cmd_rep`	291	0.90	+4.1pp
	`file_diff`	108	0.83	−2.9pp
	`outcome_bp`	42	0.72	−13.9pp
	multi-signal	44	0.94	+8.1pp
Outcome	Resolved	8	0.94	+8.1pp
	Unresolved	477	0.85	−0.9pp

structural classifiers performing best (F1 = 0.83–0.88).

Semantic residual (28.9%). The remaining 78 windows (28.9% of gold stagnation) constitute the *semantic residual*—stagnation that no current automated method captures effectively:

- **Strategic dead-ends** (11.5%): The agent pursues a coherent but fundamentally wrong approach. Commands are diverse, files change, no LF fires—only understanding the task goal reveals the futility.
- **Implicit repetition** (8.9%): The agent retries the same logical approach with different surface commands (e.g., `grep` → `find` → `ag`). Literal command repetition is low, but intent-level repetition is high.
- **Unfocused exploration** (8.5%): Rapid switching between non-viable approaches. Structurally resembles productive exploration; semantically is directionless thrashing.

All methods achieve F1 $\leq$ 0.38 on semantic residual types. The LLM-as-judge performs best on strategic dead-ends (0.38), leveraging partial intent understanding, but still fails far more than it succeeds. Probes and structural classifiers are near-blind to these categories. This 28.9% semantic residual represents the structural ceiling of current approaches and a concrete target for future task-aware methods.

8 Fine-Grained Performance Analysis

Table 9 reports stagnation F1 broken down by trajectory position, dominant LF, and outcome on the LF test set (N = 1,824, conf$\geq$2).

Three patterns emerge. First, **mid-trajectory stagnation is hardest** (F1 = 0.79), reflecting ambiguous transitional states where stagnation onset/offset boundaries are gradual rather than crisp. Second, **`outcome_bp`-dominated windows are the weakest** (F1 = 0.72) because this lagging signal fires well after stagnation begins, creating label boundary noise. Third, **multi-signal agreement yields the cleanest labels** (F1 = 0.94), confirming that LF consensus is a reliable quality indicator.

On the gold set, the structural–semantic split is stark: structural stagnation achieves probe F1 = 0.78, while semantic stagnation drops to 0.27—a 51pp gap that underscores the fundamental limitation of training on LF-derived labels for detecting non-structural failure modes.

9 Temporal Label Models: A Well-Motivated Negative Finding

The systematic temporal dynamics of LFs (§6) motivate temporal label models (TLMs) that explicitly model LF firing sequences rather than treating each window independently.

9.1 Implementations

- **TWV (Temporally Weighted Voting)**: Gaussian-weighted votes where recent LF firings receive higher weight. Controlled by bandwidth σ.
- **HMM-TLM**: Hidden Markov Model with latent stagnation/progress states and LF emissions, modeling transition dynamics.
- **Lag-aware HMM**: HMM variant incorporating learned per-LF lag offsets to align temporally shifted signals.

9.2 Results

TWV is harmful for the vast majority of σ values. The only weakly positive configuration ($\sigma = 2$) produces labels that, when used to train downstream structural classifiers, degrade performance across all evaluation settings: in-domain −4.9pp, cross-agent −21.7pp, conf≥2 −1.8pp. HMM-TLM overpredicts the stagnation class. Lag-aware HMM matches but does not exceed majority vote (F1 = 0.894 vs. MV 0.894).

9.3 Failure Analysis

The failure of all three TLM variants traces to a common root cause: the low-conflict regime of the current 3-LF framework. At conf≥2, LFs agree 95.1% of the time, leaving minimal room for temporal re-weighting to improve. TWV's primary effect is changing labels in the low-confidence (conf=1) stratum, but it changes them in the wrong direction—introducing noise rather than correcting errors. This negative finding provides an important constraint: temporal label models require higher LF diversity and disagreement rates to offer benefits over simple majority vote.

10 Discussion

Reinterpreting "structural dominance." Our gold-validated results support a nuanced conclusion: LF-approximated stagnation is structural in nature, and structural features remain the best practical choice for detection given their efficiency (<5ms) and cross-domain robustness. However,

human-recognizable stagnation has additional semantic dimensions that current NL methods—including neural probes and LLM-as-judge—have not yet effectively captured. We do not claim that semantic understanding is valueless, but rather that current automated approaches to encoding semantics fail to capture the task-level reasoning required to detect strategic dead-ends, implicit repetition, and unfocused exploration.

Circular evaluation as a methodological warning. The ∼22pp inflation from LF-eval to gold-eval is a cautionary finding for any weak supervision study. When structural features serve as inputs to both the labeling functions and the downstream classifier, near-perfect LF-eval scores reflect rule reconstruction, not genuine capability. We recommend that studies using programmatic labeling always report the circular evaluation gap alongside headline numbers.

Practical implications. For deployment, structural classifiers offer the best cost–performance tradeoff: lightweight, fast, and outperforming alternatives on both LF-derived and human-validated labels. The 28.9% semantic residual represents a known blind spot that should be disclosed when deploying such monitors. For settings where this residual is critical, ensemble approaches combining structural features with targeted LLM evaluation of low-confidence windows may offer a practical compromise.

11 Limitations

- **Domain scope.** All data comes from software engineering agents (SWE-bench). While structural stagnation patterns (command repetition, inactivity) likely generalize, the specific LF thresholds and taxonomy categories may not transfer to non-SE domains.
- **Agent scope.** Cross-agent evaluation covers only two agents (SWE-Agent, OpenHands). The `cmd_rep` role reversal on OpenHands demonstrates that findings are agent-specific; broader agent coverage is needed.
- **Gold set size.** The 650-window gold set, while sufficient for the comparisons reported, limits statistical power for fine-grained subgroup analyses (e.g., $n = 23$ for unfocused exploration).
- **LLM-as-judge.** We evaluate a single LLM (Claude Sonnet) in a zero-shot setting. Few-shot or fine-tuned LLM judges may perform better, though at higher cost.
- **Temporal noise.** Onset alignment (±2–3 steps) reflects both genuine measurement noise and the inherent subjectivity of stagnation onset boundaries.

12 Conclusion

We present a systematic empirical study of progress stagnation detection in LLM agents, validated against 650 human-annotated windows. Our findings deliver strong empirical evidence with identified limitations: structural features dominate all current automated approaches (gold macro-F1 = 0.76 vs. 0.72 for the best probe and 0.60 for LLM-as-judge), but 28.9% of human-recognized stagnation constitutes a semantic residual beyond their reach. LF temporal dynamics provide useful coarse-grained onset signals (±2–3 step resolution), and temporal label models fail to improve over majority vote in the current low-conflict LF regime.

These results redirect agent monitoring research along two axes: toward lightweight structural approaches for practical deployment, and toward task-aware semantic methods for the identified

residual. The 28.9% semantic ceiling—comprising strategic dead-ends, implicit repetition, and unfocused exploration—provides a concrete benchmark for future progress.

References

J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Xia, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *NeurIPS*, 2024.

X. Wang, B. Li, Y. Song, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré. Snorkel: Rapid training data creation with weak supervision. *VLDB*, 2017.

Z. Xie et al. AgentPRM: Process reward model for agent training via TD and GAE. *ACM Web Conference*, 2026.

N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 2023.

A. Zhou, Y. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang. Language agent tree search unifies reasoning acting and planning in language models. *ICML*, 2024.

S. Yao, D. Yu, J. Zhao, et al. Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*, 2023.

Y. Jiang et al. SlopCodeBench: Benchmarking coding agent degradation in extended contexts. *arXiv preprint arXiv:2603.24755*, 2026.

C. E. Jimenez, J. Yang, A. Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? *ICLR*, 2024.

X. Chen et al. Monitoring LLM agents for sequentially contextual harm. *ICLR Workshop*, 2025.